# CS6140 Final Project: Airbnb Price Modeling

Vaibhav Thalanki, Ananya Hegde, Rahul Kulkarni

**Abstract**

Short-term rental platforms such as Airbnb rely heavily on accurate pricing strategies to balance demand and maximize revenue for hosts. However, determining an appropriate listing price is challenging because it depends on many factors including property characteristics, location, host attributes, and user reviews. In this project, we built machine learning models that predict the log-price of Airbnb listings based on these features. Using the "Airbnb Listings in Major U.S. Cities" dataset from Kaggle — 74,111 listings across six U.S. cities — we analyzed how different listing attributes influence pricing and trained five regression models spanning linear, kernel, tree-ensemble, and neural families to estimate listing prices. We preprocessed the dataset, handled missing values, encoded categorical variables, and engineered additional features including Haversine distances to city landmarks, VADER sentiment on listing text, per-city temporal signals recovered from review timestamps, and a 128-dimensional multi-hot encoding of amenities, yielding a 206-feature matrix. On a held-out 80/20 validation split, **XGBoost** achieved the best performance with **RMSE 0.334** and $\mathbf{R^2}$ **0.784**, outperforming HistGradientBoosting, a custom 8-layer ANN, Linear Regression, and Linear SVR. The results can help hosts set competitive prices and provide insights into the factors that influence Airbnb pricing.

**Project Repository on GitHub**

## 1 Introduction

Pricing short-term rental properties on platforms such as Airbnb is a complex task influenced by numerous factors including property characteristics, neighborhood location, availability, host reputation, and customer reviews. Identifying the optimal price for a listing is difficult because the relationship between these features and price is often nonlinear and influenced by multiple interacting variables.

This project addresses the problem of predicting Airbnb listing prices using machine learning techniques. By analyzing historical Airbnb data from major U.S. cities, we aim to identify patterns between listing attributes and their associated prices and build predictive models that can estimate the price of a listing based on its features.

Accurate pricing can help hosts increase occupancy rates and maximize revenue while also improving transparency for travelers by helping them understand whether a listing is fairly priced relative to similar properties.

However, this problem presents several challenges. Airbnb datasets often contain missing values, categorical attributes such as room types and neighborhoods, and significant variation

in prices across different cities. Addressing these challenges requires careful preprocessing, feature engineering, and model selection. Applications of this work include dynamic pricing systems, recommendation engines for rental platforms, and real estate market analysis.

## 2 Technical Approach

The end-to-end pipeline — from raw CSVs through preprocessing, feature engineering, split, model training, and validation — is summarized in Figure 1.

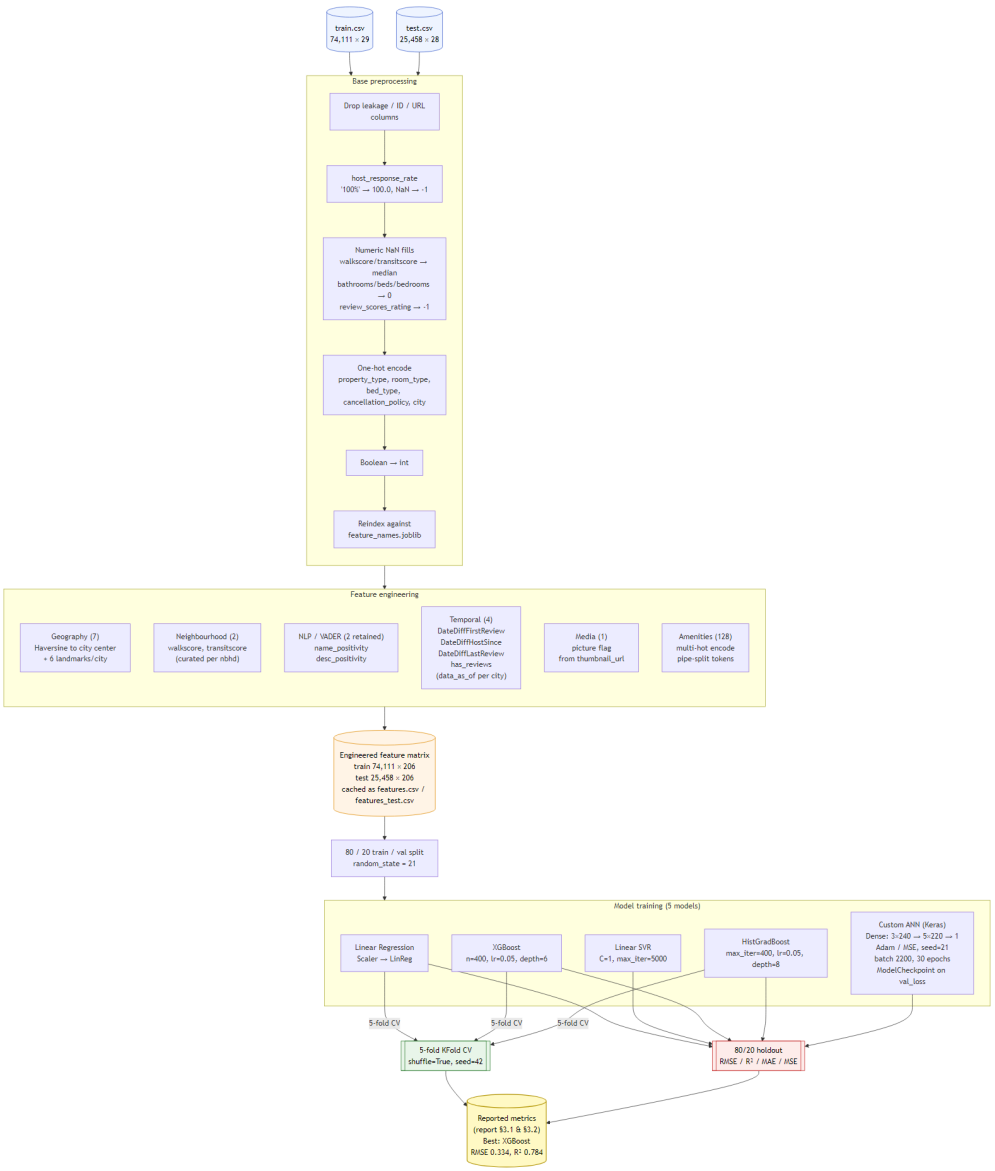


Figure 1: End-to-end pipeline

## 2.1 Dataset

We use the Airbnb Listings in Major U.S. Cities dataset released by Deloitte on Kaggle for their ML Competition [1]. The task is framed as a regression problem on $log_price$ (natural log of listing price in USD). The test set has no ground-truth $log_price$ (it was the Kaggle competition holdout), so all reported metrics are on an 80/20 held-out split of the training data ($random_state$=21). Figure 3 shows 1000 random Airbnb listings in Boston.

- Train: 74,111 rows, 29 columns (includes log_price).
- Test: 25,458 rows, 28 columns (no log_price).

Raw column types: 1 boolean, 7 float, 3 integer, 18 object/string. Features span property attributes (rooms, bathrooms, accommodates, bed type, room type, property type), geography (city, neighbourhood, lat/long, zipcode), host profile (response rate, identity verified, host since, picture), reviews (count, first/last review dates, rating), text (name, description, amenities, thumbnail URL), and policy (cancellation, cleaning fee, instant bookable). There are 6 cities covered which include Boston, New York, Los Angeles, San Francisco, Chicago and Washington D.C. The target distribution, that is, $log_price$ is approximately normal, mean $\approx$ 4.78, std $\approx$ 0.72 (see Fig. 2).

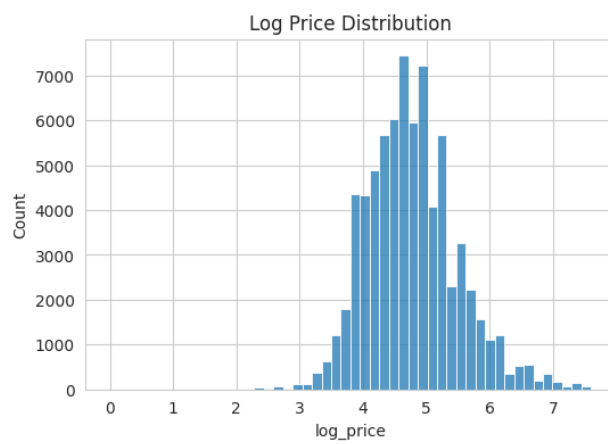


Figure 2: Log Price Distribution

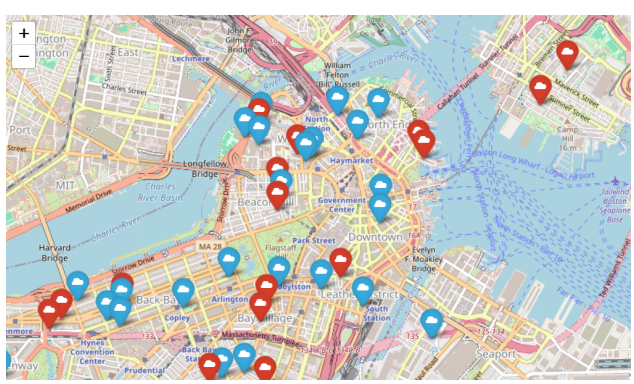

Figure 3: Random 1000 train & test samples in Boston

## 2.2 Feature Engineering

Feature engineering is performed in *Airbnb_modeling_CS6140_ML.ipynb* and cached to *data/engineered/features.csv* and *features_test.csv*, refer Table 1. The final feature matrix is (74111, 206) for train and (25458, 206) for test after one-hot expansion.

### 2.2.1 Base preprocessing

- Drop leakage-prone / irrelevant columns (raw ID, URL strings other than thumbnail_url, etc.).
- Normalize host_response_rate: "100%" → 100.0; NaN → -1 sentinel.
- Median/sentinel fills for remaining numeric NaNs (walkscore/transitscore → median; bathrooms/beds/bedrooms → 0; review_scores_rating & DateDiffHostSince → -1).
- One-hot encoding for nominal categoricals: property_type, room_type, bed_type, cancellation_policy, city.
- Boolean → int.
- Reindex against a canonical *feature_names.joblib* so train and test matrices are byte-aligned.

### 2.2.2 Engineered Features

| **Group** | **Added features** | **Count** |
|---|---|---|
| Geography | Haversine distance to city-center + 6 curated tourist attractions per city | 7 |
| Neighbourhood | Manually curated `walkscore`, `transitscore` per neighbourhood | 2 |
| Text / NLP | VADER sentiment on `name` and `description` (pos/neg/neu/compound × 2) | 8 computed; 2 retained (`name positivity`, `desc positivity`) |
| Temporal | `DateDiffFirstReview`, `DateDiffHostSince`, `DateDiffLastReview`, `has_reviews`; reference date `data_as_of` inferred per-city as max(`last_review`) | 4 |
| Media | `picture` binary flag derived from presence of `thumbnail_url` | 1 |
| Amenities | Raw {...} string cleaned and pipe-split; multi-hot encoded | 128 |

Table 1: Feature groups and engineered features

**Haversine distances**
For many people, the important part of the location of each listing is it's distance to the City Center, as well as the proximity to main touristic attractions. For this, We have googled the cooridanates for each city center as well as their main attractions. We went with using "km" instead of miles as it gives larger numbers and thus hopefully more precision.

Six landmarks per city are hardcoded with lat/long (e.g. for NYC: Times Square, Empire State, Statue of Liberty, Central Park, Rockefeller Center, Lower Manhattan; for Boston: Faneuil Hall, Fenway Park, Harvard, MIT, Old North Church, South Boston), plus a downtown anchor. Distances are computed with the standard spherical formula, Earth radius 6371 km.

**VADER sentiment [2]**
VADER converts 'name' and 'description' of Airbnb listings into four numeric sentiment scores (positivity, neutrality, negativity, compound) that summarize tone and can be used as compact predictors in regression.

Applied to both name and description, producing four polarity scores per field. Only the positivity scores were retained in the modeling set based on correlation analysis.

**Temporal reference.**
The Kaggle release stripped scrape dates, so we recover an effective data_as_of per city as train.groupby('city')['last_review'].max(). We are basically using the most recent review date observed per city, which is a well-known trick for inferring the scrape date of an Airbnb dump. This gives LA/Chicago/DC scrape ≈ May 2017 and NYC/SF/Boston scrape ≈ October 2017. All host-tenure and review-recency features are expressed as days relative to this reference.

*Why it matters for price?*

- Experienced hosts (high DateDiffHostSince) tend to price more confidently and often higher.
- Long-running listings (high DateDiffFirstReview) have established reputations.
- Recently reviewed listings (low DateDiffLastReview) are active/in demand — stale listings often indicate abandonment or low demand, correlating with lower price.

**Amenities**
The raw amenities field (e.g. "TV,Wifi,Äir conditioning;...") is stripped of braces and quotes, comma-split, pipe-joined, and multi-hot encoded into 128 boolean columns — one per unique amenity observed across the full train+test union.

## 2.3 Model Selection

Per the course requirement of three or more models including an ANN, we trained five regression models spanning linear, kernel-approximated, tree-ensemble, and neural families, refer Table 2.

| Model | Library | Key hyperparameters |
|---|---|---|
| Linear Regression | scikit-learn | Pipeline(StandardScaler → LinearRegression), defaults |
| Linear SVR | scikit-learn | $C = 1.0$, max_iter=5000, on scaled features |
| XGBoost | xgboost 2.x | n_estimators=400, learning_rate=0.05, max_depth=6, random_state=42 |
| HistGradientBoosting | scikit-learn | max_iter=400, learning_rate=0.05, max_depth=8, random_state=42 |
| Custom ANN | Keras / TensorFlow | see below for ANN Definition |

Table 2: Models used and their key hyperparameters

**Cross-validation**
Linear Regression, XGBoost and HistGradBoost were evaluated with 5-fold KFold CV (shuffle=True, random_state=42) on the full 74,111-row training set. LinearSVR was excluded from CV due to convergence cost at this data size (5000-iter liblinear), and the ANN was excluded because cross-validated deep-learning runs were outside the compute budget for a CPU-only setup. Both were evaluated on the same 80/20 holdout used for the final comparison.

## 2.4 ANN Definition

A deep feed-forward network built in Keras Sequential API. It is chosen to be just wide enough to exploit the 206-dim feature space without overfitting and shallow enough to train on CPU in under 5 minutes, refer Table 3. The checkpointed best model (≈ epoch 12) is serialized to *saved/models/model_ann.keras* in the Keras 3 native format.

```
tf.random.set_seed(21); np.random.seed(21)
ann = Sequential([
    Dense(240, activation='relu', input_dim=206),
    Dense(240, activation='relu'),
    Dense(240, activation='relu'),
    Dense(220, activation='relu'),
    Dense(220, activation='relu'),
    Dense(220, activation='relu'),
    Dense(220, activation='relu'),
    Dense(220, activation='relu'),
    Dense(1, activation='relu'),
])
ann.compile(optimizer='adam', loss='mean_squared_error', metrics=['
   mae'])
```

| Property | Value |
|---|---|
| Input dim | 206 |
| Hidden layers | 8 Dense ($3 \times 240 \rightarrow 5 \times 220$), all ReLU |
| Output | 1 unit, ReLU (prices are strictly positive on log scale within our support) |
| Total trainable params | 413,081 ($\approx$ 1.58 MB) |
| Optimizer / loss / metric | Adam (defaults) / MSE / MAE |
| Batch size / epochs | 2200 / 30 |
| Regularization | None explicit; ModelCheckpoint on `val_loss` with `save_best_only=True` |
| Input normalization | `StandardScaler` fit on train fold only; persisted to `scaler_ann.joblib` |
| Seeds | TF and NumPy both set to 21 for reproducibility |

Table 3: ANN architecture and training configuration

# 3 Experimental Results

## 3.1 Cross-Validation Scores (5-fold, full train set)

| Model | RMSE (mean $\pm$ std) | $R^2$ (mean $\pm$ std) |
|---|---|---|
| Linear Regression | 0.4358 $\pm$ 0.0032 | 0.6310 $\pm$ 0.0036 |
| XGBoost | 0.3716 $\pm$ 0.0030 | 0.7315 $\pm$ 0.0041 |
| HistGradBoost | 0.3732 $\pm$ 0.0030 | 0.7293 $\pm$ 0.0043 |

Table 4: CV Model performance comparison

These are the honest generalization estimates — LinearSVR and ANN have only holdout numbers below, refer Figure 4.

## 3.2 Validation Metrics (80/20 holdout, random_state=21)

Computed from TA_evaluation_run.ipynb against 14,823 held-out listings. The holdout numbers for the three CV'd models are slightly better than their CV means because the saved artifacts were refit on the full 80% train partition after CV — a fair post-selection fit, but they no longer carry cross-validated uncertainty.

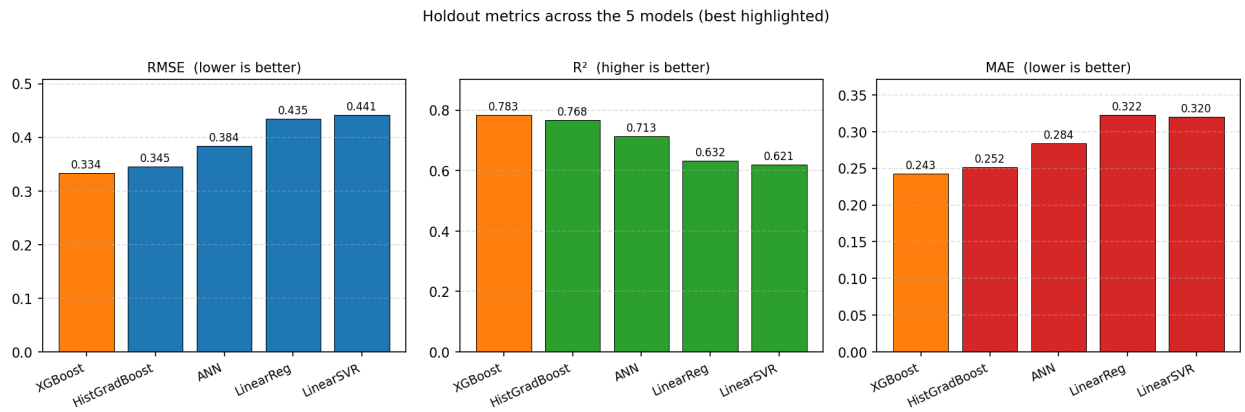


Figure 4: Holdout Metrics for models

## 3.3 Test-Set Prediction Statistics

As the dataset is part of the competition, the test set has no labels, so no metrics. Reported for distributional sanity only:

| **Model** | **mean**($\hat{y}$) | **std**($\hat{y}$) |
|---|---|---|
| LinearRegression | 4.781 | 0.572 |
| XGBoost | 4.778 | 0.615 |
| HistGradBoost | 4.778 | 0.614 |
| LinearSVR | 4.754 | 0.563 |
| ANN | 4.793 | 0.628 |

Table 5: Test set performance

## 3.4 Takeaways

XGBoost is the best generalizer, winning on all four metrics and on CV mean RMSE. Tree ensembles (XGB, HGB) dominate linear models by ≈0.10 in RMSE — the price surface has strong nonlinearities (interactions between city, room_type, accommodates, and amenities) that a hyperplane cannot capture. The ANN lands between the tree ensembles and the linear models. With 413 k params and no dropout/L2, we expected overfitting, but ModelCheckpoint on val_loss caught the best epoch (≈ 12), refer Figure 5. More aggressive regularization or a wider/deeper search would likely close the gap to XGBoost. LinearSVR and LinearRegression essentially tie, as expected — both are convex surrogates of the same hyperplane family on this feature set. For the test set, all five models predict test means within 0.04 of the train mean (4.78), indicating no systematic distribution shift between splits, refer Table 5.

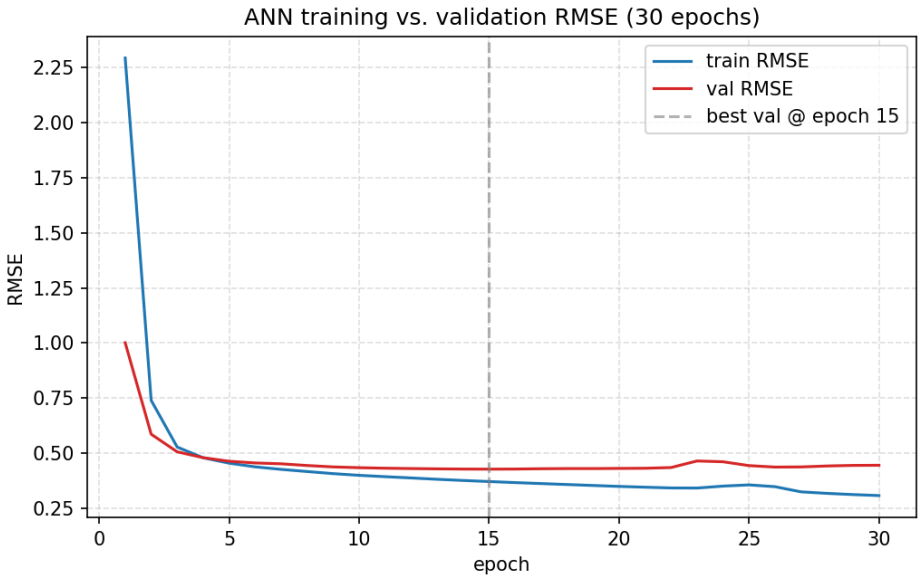


Figure 5: ANN training curve

# 4 Conclusion & Future Work

We built a reproducible pipeline for predicting Airbnb listing log-prices across six U.S. cities, covering feature engineering (geography, NLP sentiment, temporal, amenities), five regression models spanning linear, kernel, tree-ensemble, and neural families, and a single-notebook evaluation path. XGBoost is our best model at RMSE 0.334 / $R^2$ 0.784 on a 14,823-row held-out validation set, with the custom 8-layer ANN third behind HistGradBoost. All numbers satisfy the project requirement of three-plus models including an ANN — we delivered five.

The bigger-picture finding is that structured tabular pricing is still best served by tree ensembles. The ANN is competitive but not dominant, and the added engineering cost (scaling, checkpointing, Keras-specific save/load) is only worthwhile if downstream goals (e.g. joint text encoding, embedding reuse) need a neural representation.

Several feature-engineering candidates were scoped but not implemented due to time: per-city, per-month occupancy proxies derived from last_review / number_of_reviews timestamps to capture weekday/weekend and seasonal pricing swings; sentence-transformer embeddings (e.g. all-MiniLM-L6-v2) of description projected via PCA, which would likely outperform the 2-dim VADER positivity scores we kept and a scripted walkscore / transitscore pull from the Walk Score API to scale beyond our manually curated values. On the modeling side, obvious next steps are Bayesian hyperparameter search (Optuna) for XGBoost, ANN regularization (Dropout, L2, EarlyStopping with patience) and a wider architecture search and maybe a mixture-of-experts setup.

# 5 Contributions

| Member | Contributions |
|---|---|
| Vaibhav Thalanki | Feature engineering (geography/distance features, VADER sentiment, temporal `data_as_of` recovery, amenities multi-hot encoding), ANN architecture design and training and TA evaluation notebook. |
| Ananya Hegde | XGBoost and LinearSVR training and tuning, test-set evaluation, prediction-distribution sanity checks. |
| Rahul Kulkarni | Exploratory data analysis (target distribution, missingness, correlation heatmap, location plots), preprocessing pipeline (missing-value strategy, categorical encoding, column alignment), Linear Regression training. |

# References


[1] R. Mizrahi, *Airbnb Listings in Major U.S. Cities (Deloitte Machine Learning Competition)*, Kaggle Dataset, 2018. Available: `https://www.kaggle.com/datasets/rudymizrahi/airbnb-listings-in-major-us-cities-deloitte-ml/data`

[2] Hutto, C.J. & Gilbert, E.E. (2014). VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text. Eighth International Conference on Weblogs and Social Media (ICWSM-14). Ann Arbor, MI, June 2014.